Project Report

Autonomous Delivery Agent

Comparative Study of BFS, UCS, and A* in a Dynamic Grid Environment

Submitted by: AASTHA

Submitted to: MK Jayanti Mam

Registration Number: 25BCE10812

Branch: CSE Core

Course: Fundamentals of AI ML CSA2001

Date: March 31, 2026

Slot: B11+B12+B13

TITLE

A Python-based simulation that demonstrates how an autonomous delivery agent navigates a 10×10 grid environment using BFS, UCS, and A* pathfinding algorithms, with real-time replanning when dynamic obstacles block the path.

INTRODUCTION

The Autonomous Delivery Agent project simulates how an intelligent agent can find the most efficient path to a destination in a changing environment. Delivery agents — whether human couriers or autonomous robots — must navigate roads with obstacles, traffic, and unpredictable blockages.

This project models such a scenario on a 10×10 grid containing static wall obstacles and two moving obstacles. Three classical AI search algorithms — Breadth-First Search (BFS), Uniform Cost Search (UCS), and A* — are implemented and compared based on their effectiveness. The agent automatically replans its route whenever a moving obstacle blocks its current path, demonstrating real-time adaptive decision-making.

MOTIVATION OF THE PROJECT

Manual route selection is inefficient, inconsistent, and cannot adapt to sudden changes. By simulating autonomous pathfinding, this project shows how AI-driven navigation can optimize delivery routes, reduce time, minimize cost, and respond dynamically to environmental changes — capabilities that are critical for real-world logistics and robotics systems.

PROBLEM STATEMENT

Delivery agents must find the shortest and most cost-efficient path from a start point to a destination in environments where obstacles may appear or move unexpectedly. The specific challenges include:

- Finding a valid path from (0,0) to (9,9) on a 10×10 grid with static walls.
- Avoiding dynamic moving obstacles that change position each time step.
- Replanning the route in real time when the planned path becomes blocked.
- Comparing the efficiency of different pathfinding algorithms under identical conditions.

OBJECTIVE OF THE PROJECT

- Automatically simulate a dynamic grid environment with static and moving obstacles.
- Implement BFS, UCS, and A* algorithms for autonomous pathfinding.
- Enable the agent to replan its route when a moving obstacle blocks the path.
- Compare algorithm performance based on number of steps and execution time.
- Visualize the agent's movement step-by-step in the terminal.

EXISTING METHODS

1. Manual Route Selection: A human operator selects the path manually at each step.
2. Static Pathfinding (No Replanning): Path is computed once and followed blindly, even if blocked.
3. BFS / UCS / A* with Replanning: Path is computed by an algorithm and recomputed whenever the environment changes — the approach used in this project.

PROS AND CONS OF THE STATED METHODS

Method	Pros	Cons
Manual Selection	Full human understanding of context	Slow, error-prone, does not scale
Static Pathfinding	Simple to implement; fast initial plan	Fails when obstacles move; no adaptability
BFS	Guaranteed shortest path; complete	High memory usage; slow for large grids
UCS	Optimal cost path; flexible to weights	No heuristic; similar memory issues as BFS
A*	Fast with heuristic; optimal and efficient	Heuristic-dependent; more complex

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware:

- Processor: Intel Core i3 or higher
- RAM: 2 GB minimum (4 GB recommended)
- Storage: 50 MB free disk space

Software:

- Python 3.12
- IDE: VS Code / PyCharm / Google Colab
- Libraries: collections, heapq, time, copy (built-in); pandas, matplotlib (optional)
- Version Control: GitHub

METHODOLOGY AND GOAL

1. Create a 10×10 grid with static wall obstacles at predefined positions.
2. Place the agent at (0,0) and set the goal at (9,9).
3. Initialize two moving obstacles at (2,6) and (5,5).
4. Run each algorithm: compute path from current agent position to goal.
5. Move the agent one step at a time; move obstacles before each step.
6. If the next step is blocked by a moving obstacle, trigger replanning.
7. Record total steps taken and execution time for each algorithm.

Goal: Reach (9,9) in minimum steps, demonstrating adaptive real-time navigation.

FUNCTIONAL MODULES DESIGN AND ANALYSIS

1. Grid Module (create_grid): Builds the 10×10 grid and places static walls at (1,3)–(4,3), (6,6)–(6,8), (7,8).

2. Visualizer Module (print_grid): Renders the grid in the terminal, marking S (Start), G (Goal), A (Agent), 1 (Wall), M (Moving Obstacle).

3. Validator Module (is_valid): Checks boundary limits, wall collision, and moving obstacle collision before any move.

4. Neighbor Module (get_neighbors): Returns valid adjacent cells (Up, Down, Left, Right) for a given node.

5. Pathfinding Module (bfs / ucs / a_star): Three interchangeable algorithm implementations sharing the same interface.

6. Obstacle Module (move_obstacles): Moves dynamic obstacles left or right each time step.

7. Simulation Module (simulate_algorithm): Orchestrates the full run — planning, stepping, replanning, and reporting.

ALGORITHM DEVELOPMENT

Algorithm for Autonomous Delivery Navigation:

8. Accept start (0,0), goal (9,9), grid, and initial moving obstacle positions.
9. Call the selected algorithm (BFS / UCS / A*) to find a path from the current position.
10. For each step in the planned path:
    - Move all dynamic obstacles one position (left-right patrol).
    - If the next step is now occupied by an obstacle, discard path and go to Step 2.
    - Otherwise, move the agent to the next step and print the grid.
11. Repeat until the agent reaches the goal.
12. Record total steps and time taken; display performance summary.

Heuristic used in A* (Manhattan Distance):

```
heuristic(a, b) = |a[0] - b[0]| + |a[1] - b[1]
```

SOFTWARE ARCHITECTURAL DIAGRAM

INPUT LAYER main.py is executed; start, goal, and obstacle positions are defined
ENVIRONMENT LAYER create_grid() → static walls placed → moving_obstacles initialized
ALGORITHM LAYER simulate_algorithm() calls bfs() / ucs() / a_star()
REPLANNING LAYER move_obstacles() → path blocked check → replan if needed
OUTPUT LAYER print_grid() at each step → steps count → time taken → Goal reached!

CODING

The project is implemented in Python 3.12 using only built-in libraries. Key components:

- create_grid() — builds the 10×10 grid array with static walls
- bfs() / ucs() / a_star() — three pathfinding algorithm implementations
- move_obstacles() — updates obstacle positions each time step
- simulate_algorithm() — runs the full agent simulation with replanning

Core A* logic:

```
pq = [(0, start)]
while pq:
    _, current = heapq.heappop(pq)
    if current == goal: return reconstruct_path(parent, goal)
    for neighbor in get_neighbors(grid, current, moving_obs):
        tentative_g = g_cost[current] + 1
        f = tentative_g + heuristic(neighbor, goal)
        heapq.heappush(pq, (f, neighbor))
```

Replanning trigger:

```
moving_obstacles = move_obstacles(moving_obstacles, grid)
if next_step in moving_obstacles:
    print('Path blocked! Replanning...')
    break  # exit inner loop → outer loop replans
```

OUTPUT

Each algorithm produces terminal output showing the grid state at every step. Example:

```
==============================
Running BFS
==============================
BFS planned path: [(0,0),(0,1),(0,2),(1,2),(2,2)...]
Current Grid:
A 0 0 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0
0 0 0 1 0 0 M 0 0 0
0 0 0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 M 0 0 0 0
0 0 0 0 0 0 1 1 1 0
0 0 0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 G
BFS: Goal reached in 18 steps!
BFS: Time taken = 2.1345 seconds
```

Results Summary:

Algorithm	Steps to Goal	Observation
BFS	18	Shortest path; explores broadly

UCS	18	Same as BFS on uniform cost grid
A*	18	Fastest; guided by heuristic

KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

- 10×10 grid with configurable static wall placement.
- Two moving obstacles with left-right patrol behavior updated each step.
- Three complete pathfinding implementations (BFS, UCS, A*) with shared interface.
- Real-time terminal grid visualization after every agent move.
- Automatic replanning triggered when a dynamic obstacle blocks the next planned step.
- Performance metrics (step count and execution time) recorded per algorithm.

SIGNIFICANT PROJECT OUTCOMES

- Successfully demonstrated BFS, UCS, and A* in a dynamic 10×10 grid world.
- Agent autonomously navigated from (0,0) to (9,9) while avoiding all obstacles.
- Real-time replanning was correctly triggered on obstacle collisions.
- A* showed faster convergence due to Manhattan distance heuristic guidance.
- BFS and UCS produced equivalent results on uniform-cost grids, validating correctness.

TESTING AND REFINEMENT

- Tested all three algorithms with the same initial grid and obstacle configuration.
- Verified that replanning triggered correctly whenever a moving obstacle blocked the path.
- Tested edge cases: start equals goal, no valid path, obstacles filling all alternate routes.
- Confirmed step counts and paths are consistent across multiple runs.

PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

- Autonomous warehouse robots (e.g., Amazon Fulfillment Centers) for package routing.
- Last-mile delivery drones navigating around dynamic no-fly zones or obstacles.
- Hospital service robots delivering medicines through corridors with moving people.
- Self-driving vehicle route planning in dynamic traffic environments.
- Game AI for NPC movement and enemy pathfinding in grid-based games.

CONTRIBUTION / FINDINGS OF THE PROJECT

- Demonstrated that AI pathfinding with replanning can reliably handle dynamic environments.
- Confirmed A* is the most efficient algorithm when a good heuristic is available.
- Showed that BFS and UCS are equivalent in uniform-cost grids, validating theory.
- Provided a clean, modular Python framework reusable for different grid layouts.

LIMITATION / CONSTRAINTS OF THE SYSTEM

- Grid is fixed at 10×10; larger environments require further optimization.
- Moving obstacles follow a simple left-right pattern only; complex movement unsupported.
- No graphical interface — visualization is terminal text only.
- All moves have equal cost (1); weighted terrain is not supported.
- Only single-agent navigation is implemented.

CONCLUSIONS

The Autonomous Delivery Agent project successfully demonstrates how classical AI pathfinding algorithms can be applied to dynamic real-world navigation problems. By implementing BFS, UCS, and A* in a grid environment with moving obstacles, the project shows that intelligent agents can adapt to changing conditions and reliably reach their goal.

The comparison reveals A* as the most efficient choice when a suitable heuristic exists, while BFS and UCS remain valid options in simpler or weighted environments. The modular design makes this project an excellent foundation for further development into GUI-based or real-world navigation systems.

FUTURE ENHANCEMENTS

- Add a graphical interface using Tkinter or Pygame for animated path visualization.
- Introduce weighted roads to simulate traffic conditions and variable travel costs.
- Implement more realistic obstacle movement (diagonal, random, patrol patterns).
- Expand to multiple delivery agents with collision avoidance between agents.
- Integrate real-world map data via OpenStreetMap or Google Maps API.
- Add performance analytics export (CSV / Excel) for detailed comparison charts.
- Implement D* Lite algorithm for more efficient dynamic replanning.

REFERENCES

13. Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
14. Python Software Foundation. Python 3.12 Documentation. https://docs.python.org/3/
15. GeeksforGeeks. BFS, UCS, and A* Algorithm Explanations. https://www.geeksforgeeks.org
16. Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics.
17. Wikipedia. A* Search Algorithm. https://en.wikipedia.org/wiki/A*_search_algorithm
18. GitHub Repository: Autonomous Delivery Agent. https://github.com/your-username/autonomous-delivery-agent